Лабораторная работа

Номер 3

Кобзев Д. К.

Содержание

1 Цель работы **5**

2 Выполнение лабораторной работы **6**

3 Выводы **15**

Список литературы **16**

Список иллюстраций

2.1 Создание простейшей топологии сети 7
2.2 Информация о хостах 8
2.3 lab_iperf3_topo2.py 9
2.4 Скрипт с функциями ограничения производительности и изоляции 10
2.5 Копия скрипта lab_iperf3_topo2.py 11
2.6 lab_iperf3.py 12
2.7 Построение графиков 13
2.8 Makefile 14

Список таблиц

1 Цель работы

Целью данной работы является знакомство с инструментом для измерения пропускной способности сети в режиме реального времени — iPerf3, а также получение навыков проведения воспроизводимого эксперимента по измерению пропускной способности моделируемой сети в среде Mininet.

2 Выполнение лабораторной работы

В каталоге /work/lab_iperf3 для работы над проектом создаем подкаталог lab_iperf3_topo и копируем в него файл с примером скрипта mininet/examples/emptynet.py, описывающего стандартную простую топологию сети mininet. Запускаем скрипт создания топологии lab_iperf3_topo.py. После отработки скрипта смотрим элементы топологии и завершаем работу mininet(Рис. 1.1).

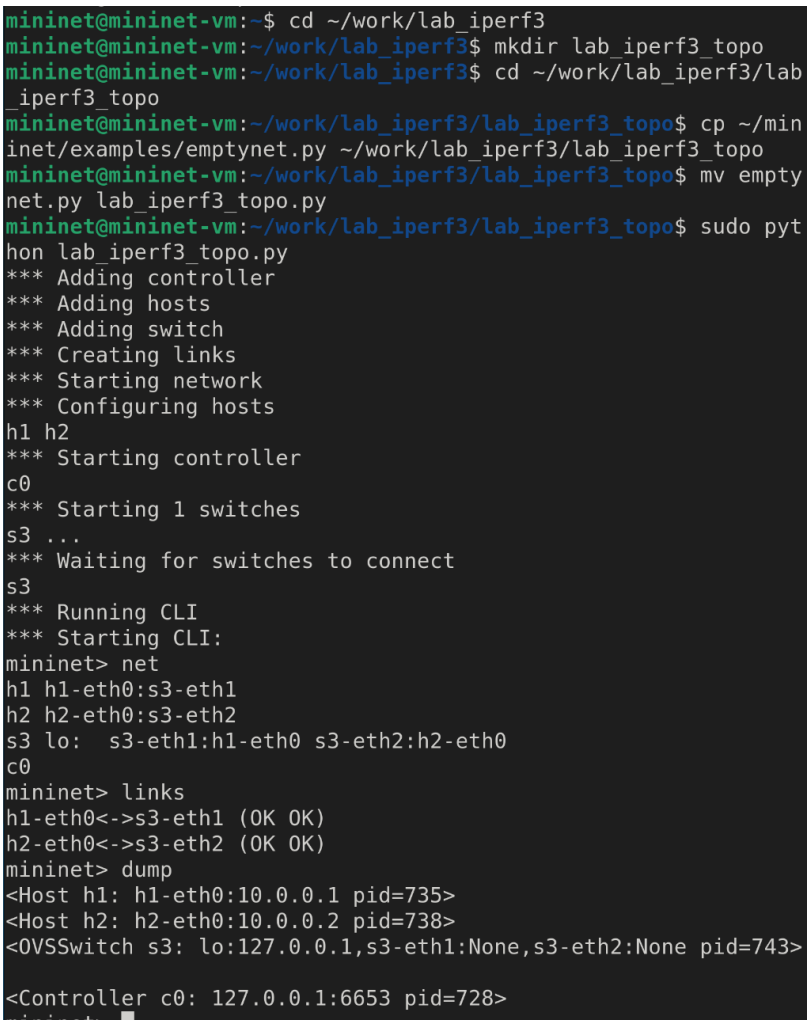


Рис. 2.1: Создание простейшей топологии сети

Вносим в скрипт lab_iperf3_topo.py изменение, позволяющее вывести на экран информацию о обоих хостах. Проверяем корректность отработки изменённого скрипта (Рис. 1.2).

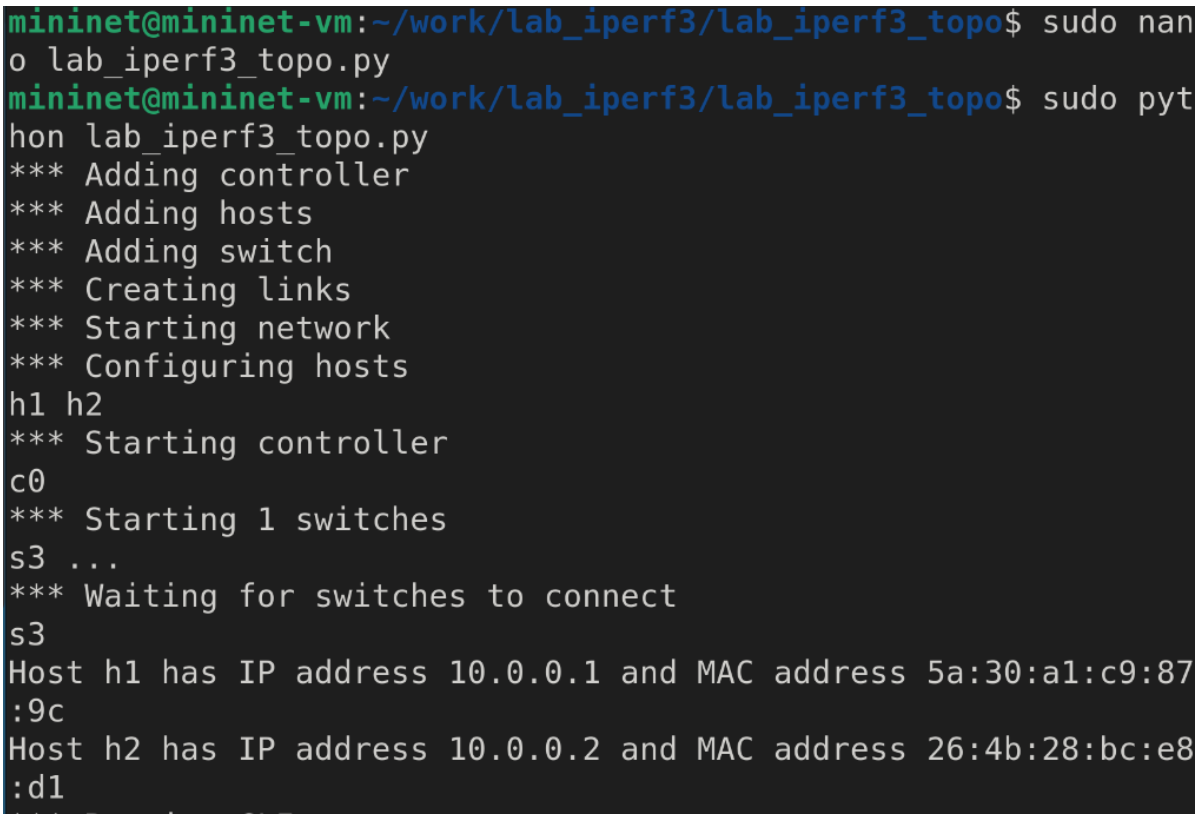
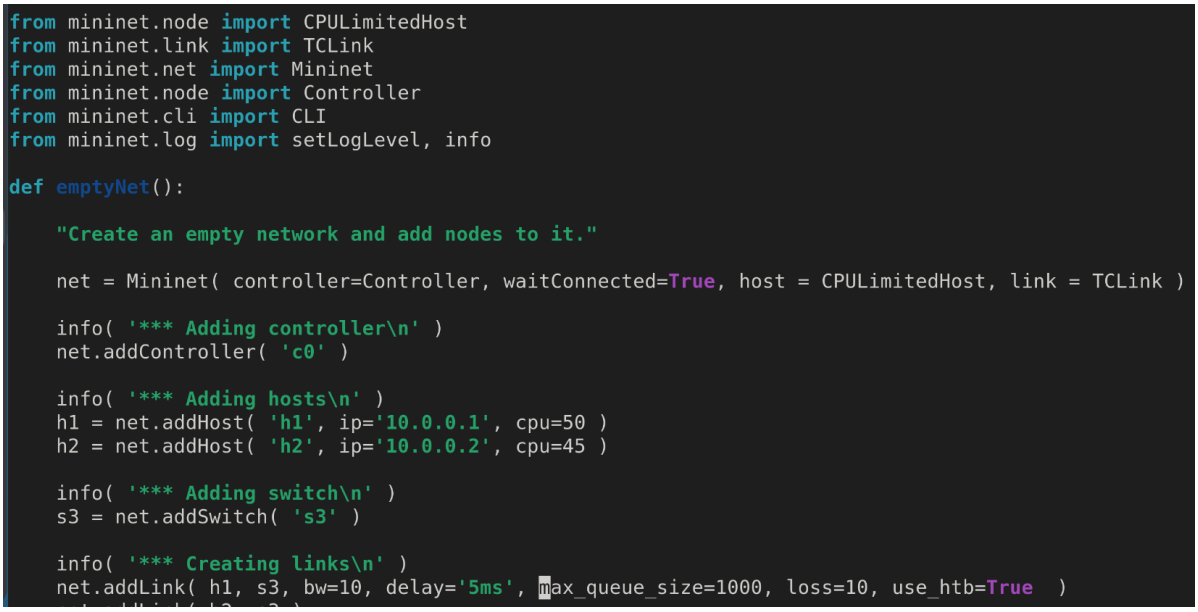


Рис. 2.2: Информация о хостах

Делаем копию скрипта lab_iperf3_topo.py. В начале скрипта lab_iperf3_topo2.py добавляем записи об импорте классов CPULimitedHost и TCLink. Изменяем строку описания сети, указав на использование ограничения производительности и изоляции. Изменяем функцию задания параметров виртуального хоста h1, указав, что ему будет выделено 50% от общих ресурсов процессора системы. Аналогичным образом для хоста h2 задаем долю выделения ресурсов процессора в 45%. Изменяем функцию параметров соединения между хостом h1 и коммутатором s3 (Рис. 1.3), (Рис. 1.4).



```
from mininet.node import CPULimitedHost
from mininet.link import TCLink
from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info

def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True, host = CPULimitedHost, link = TCLink )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1', cpu=50 )
    h2 = net.addHost( 'h2', ip='10.0.0.2', cpu=45 )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3, bw=10, delay='5ms', max_queue_size=1000, loss=10, use_htb=True  )
```

Рис. 2.3: lab_iperf3_topo2.py

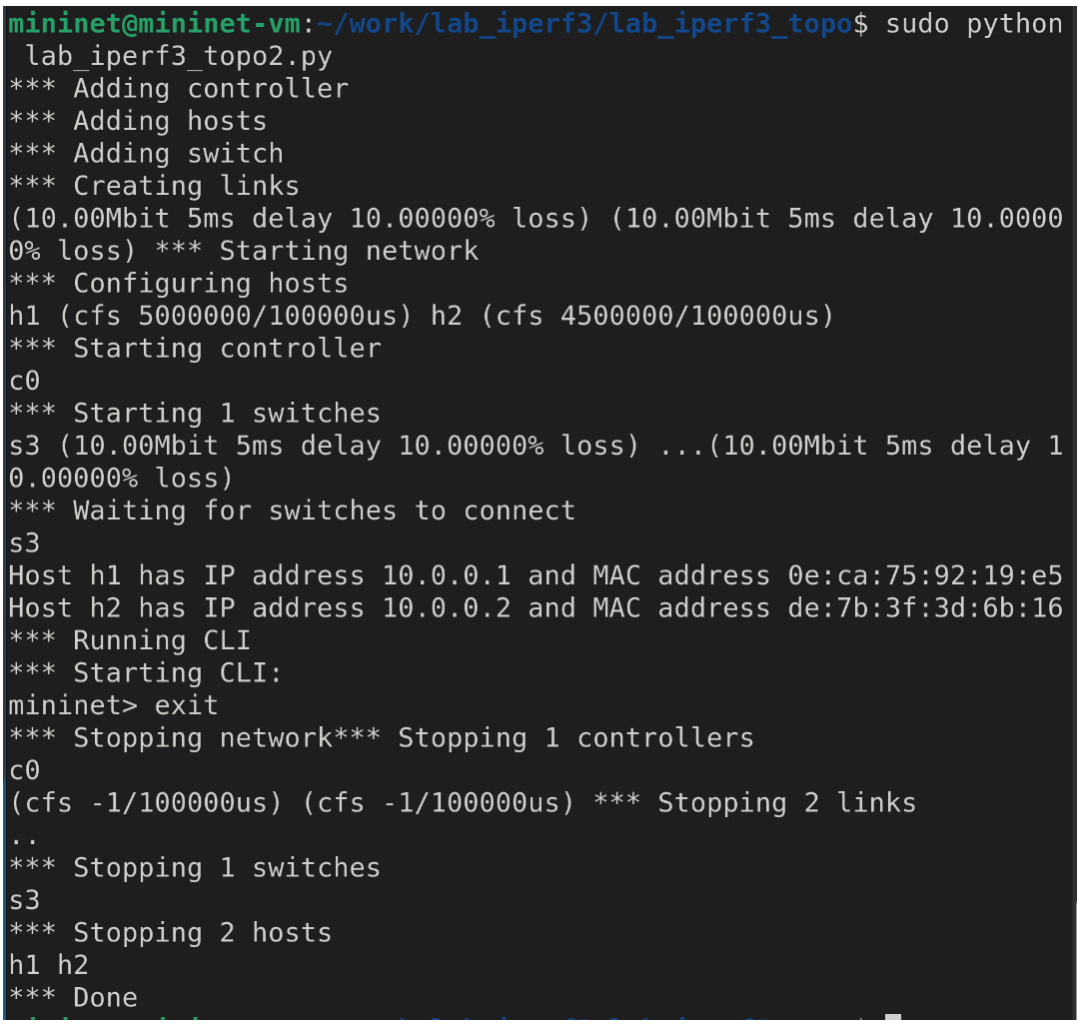


Рис. 2.4: Скрипт с функциями ограничения производительности и изоляции

Строим графики по проводимому эксперименту. Делаем копию скрипта lab_iperf3_topo2.py и поместите его в подкаталог iperf (Рис. 1.5).

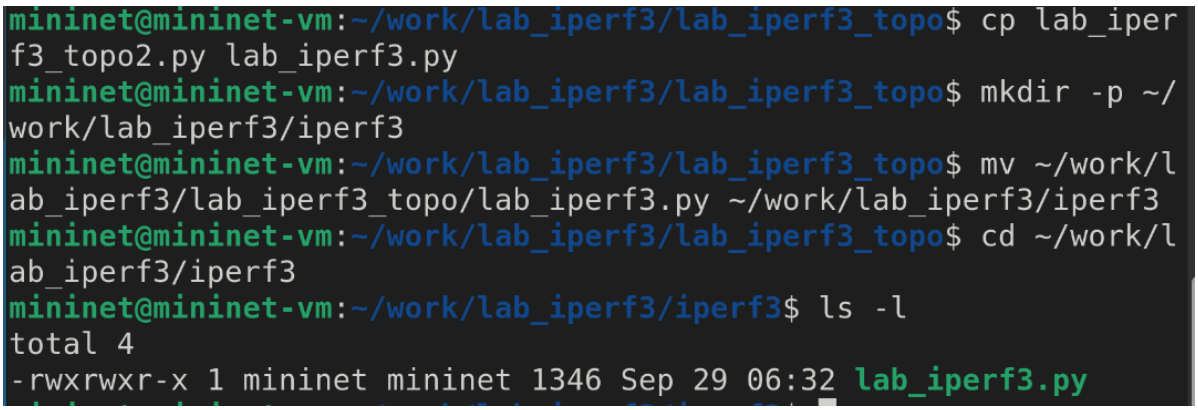


Рис. 2.5: Копия скрипта lab_iperf3_topo2.py

Изменяем код в скрипте lab_iperf3.py так, чтобы: – на хостах не было ограничения по использованию ресурсов процессора; – каналы между хостами и коммутатором были по 100 Мбит/с с задержкой 75 мс, без потерь, без использования ограничителей пропускнойспособности и максимального размера очереди. – После функции старта сети описываем запуск на хосте h2 сервера iPerf3, а на хосте h1 запуск с задержкой в 10 секунд клиента iPerf3 с экспортом результатов в JSON-файл, комментируем строки, отвечающие за запуск CLI-интерфейса (Рис. 1.6).

```
h1 = net.addHost( 'h1', ip='10.0.0.1')
h2 = net.addHost( 'h2', ip='10.0.0.2')

info( '*** Adding switch\n' )
s3 = net.addSwitch( 's3' )

info( '*** Creating links\n' )
net.addLink( h1, s3, bw=100, delay='75ms', loss=0)
net.addLink( h2, s3, bw=100, delay='75ms', loss=0 )

info( '*** Starting network\n')
net.start()
print( "Host", h1.name, "has IP address", h1.IP(), "and MAC ad
print( "Host", h2.name, "has IP address", h2.IP(), "and MAC ad

info( '*** Traffic generation\n' )
h2.cmdPrint( 'iperf3 -s -D -1' )
time.sleep(10) # Wait 10 seconds for servers to start
h1.cmdPrint( 'iperf3 -c', h2.IP(), '-J > iperf_result.json' )

#info( '*** Running CLI\n' )
#CLI( net )
```

Рис. 2.6: lab_iperf3.py

Запускаем на отработку скрипт lab_iperf3.py. Строим графики из получившегося JSON-файла. Создаем Makefile для проведения всего эксперимента (Рис. 1.7).

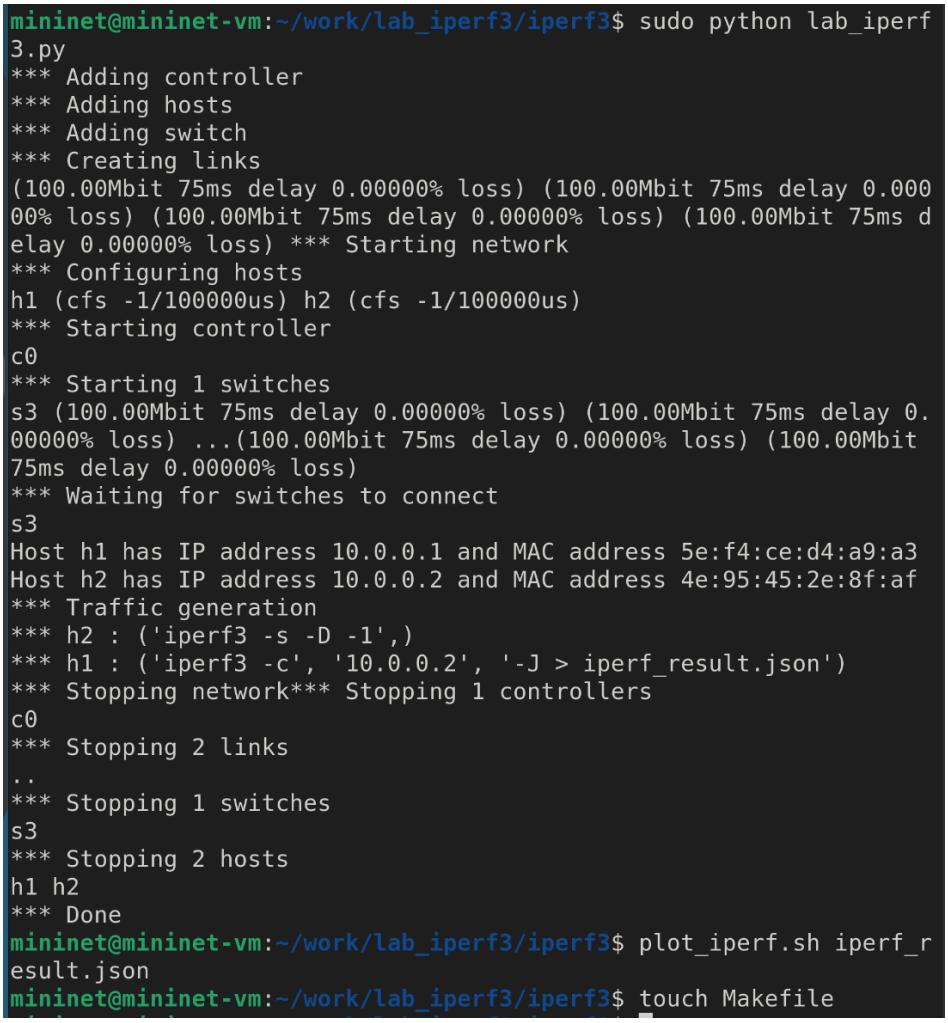


Рис. 2.7: Построение графиков

В Makefile прописываем запуск скрипта эксперимента, построение графиков и очистку каталога от результатов. Проверяем корректность отработки Makefile. Завершаем соединение с виртуальной машиной mininet и выключите её (Рис. 1.8).

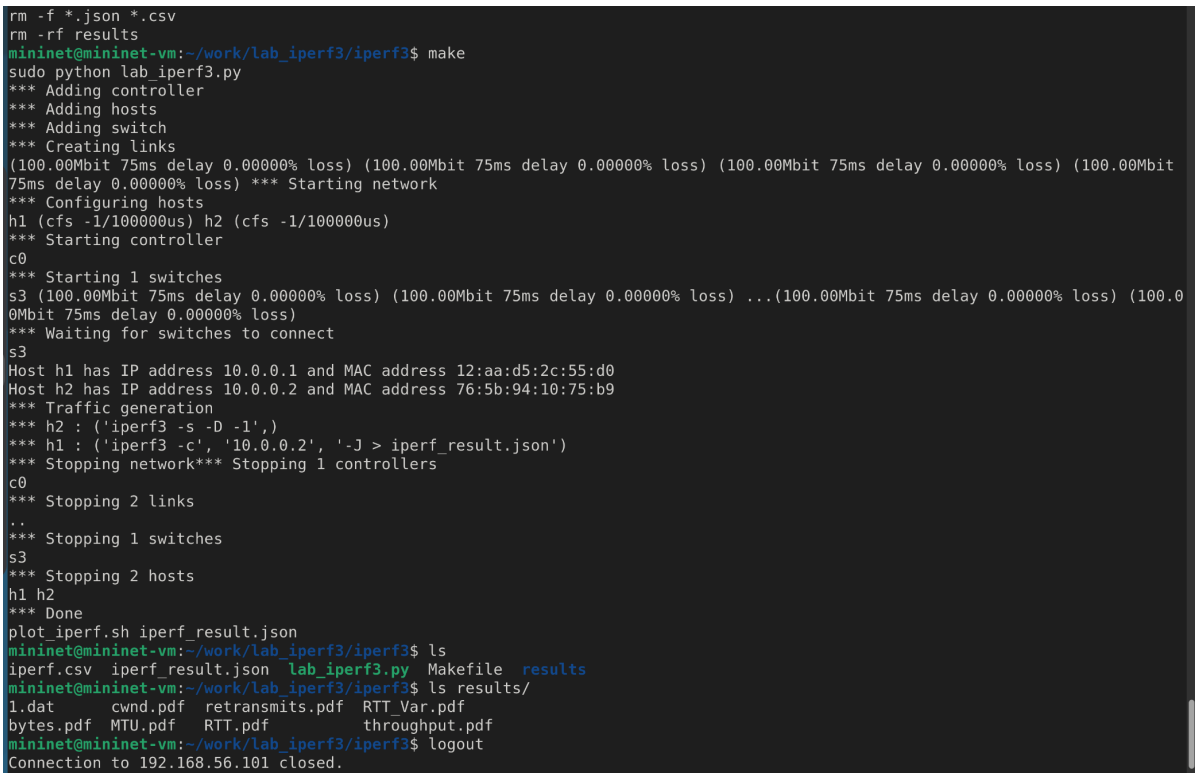


Рис. 2.8: Makefile

3 Выводы

В результате выполнения лабораторной работы я был знакомлен с инструментом для измерения пропускной способности сети в режиме реального времени — iPerf3, а также получил навыки проведения воспроизводимого эксперимента по измерению пропускной способности моделируемой сети в среде Mininet.

Список литературы